

Guía de estudio de la clase

Clase 02 - Manejo básico de Python

1. Idea Central de la Clase

La clase busca que el estudiante deje de ver Python como una lista de comandos aislados y empiece a usarlo para controlar decisiones, repetir procesos y encapsular soluciones. El aprendizaje central es pasar de escribir instrucciones sueltas a construir lógica de programación.

2. Conceptos Clave

Control de flujo

- definición clara: mecanismo que permite decidir qué parte del código se ejecuta y cuántas veces.
- explicación breve: un programa no siempre avanza en línea recta; puede bifurcarse o repetirse.
- por qué es importante: es la base para automatizar tareas.

Condicional

- definición clara: estructura que evalúa una condición y ejecuta una acción según sea verdadera o falsa.
- explicación breve: en Python se implementa con `if`, `elif` y `else`.
- por qué es importante: permite tomar decisiones programadas.

Bucle

- definición clara: estructura que repite instrucciones.
- explicación breve: `for` recorre elementos; `while` repite mientras una condición siga siendo verdadera.
- por qué es importante: evita copiar el mismo código muchas veces.

Función

- definición clara: bloque reusable de código que recibe entradas y devuelve un resultado.
- explicación breve: permite resolver una tarea específica con nombre propio.
- por qué es importante: organiza el código y mejora la reutilización.

3. Desarrollo de los Contenidos

Condicionales: decidir con reglas

La lógica de un condicional es simple: si ocurre A, haz X; si no ocurre A, haz Y. Lo importante no es memorizar la sintaxis, sino entender que el programa compara una condición booleana.

Operadores lógicos

La clase muestra que una decisión puede depender de más de una condición. Por ejemplo, pedir ingreso mínimo y antigüedad laboral. Aquí aparecen `and`, `or` y `not`, que combinan reglas simples en reglas más realistas.

Bucles: repetir sin copiar

Un `for` se usa cuando recorres una lista o una secuencia conocida. Un `while` se usa cuando la repetición depende de que algo siga ocurriendo. ELI5: `for` es como pasar lista; `while` es como seguir llenando agua mientras el vaso no esté lleno.

Funciones: convertir procedimientos en herramientas

La función toma código que se repetiría y lo encapsula. Si la función está bien escrita, se entiende qué necesita y qué entrega. Eso vuelve más legible todo el análisis posterior.

4. Ejemplos o Aplicaciones

Ejemplo 1: clasificar una nota

El condicional de notas resuelve un problema concreto: traducir una regla académica en una decisión automática. - primero se compara la nota con el umbral alto; - luego con el umbral mínimo; - finalmente se clasifica el caso restante.

Ejemplo 2: sumar ventas con `for`

El bucle recorre una lista de ventas y acumula el total. La idea central es que el mismo patrón serviría para sumar costos, visitas o cantidades.

Ejemplo 3: función `clasificar_venta`

La función recibe un monto y devuelve una etiqueta. El problema que resuelve es convertir muchos valores numéricos en categorías interpretables.

5. Errores o Confusiones Comunes

- Pensar que la indentación es opcional. En Python define los bloques de código.

- Usar `while` cuando un `for` sería más claro.
- Escribir funciones que hacen muchas cosas a la vez.
- Confundir “imprimir” un resultado con “retornarlo”. `print()` muestra; `return` devuelve.

6. Resumen Final

Esta clase instala la lógica de programación básica: decidir, repetir y reutilizar. Si el estudiante domina estos tres movimientos, podrá seguir casi todo el trabajo posterior con datos.

7. Preguntas de Repaso

1. ¿Qué diferencia hay entre `if` y `elif` ?
2. ¿Cuándo usarías `for` y cuándo `while` ?
3. ¿Qué hace `return` dentro de una función?
4. ¿Por qué la indentación cambia el significado del código?
5. ¿Qué problema resuelve una función como `clasificar_venta` ?
6. ¿Cuál es la diferencia entre `break` y `continue` ?

8. Ejercicios Propuestos

- Conceptual: explica con tus palabras cuándo un problema requiere condicionales y cuándo requiere iteración.
- Práctico: crea un programa que clasifique edades en niño, adolescente y adulto.
- Práctico: usa un `for` para contar cuántas ventas superan un umbral.
- Programación: escribe una función que reciba una lista y devuelva su promedio.